

Partie 8 : API REST

8. API REST

Définition:

Une API REST (Representational State Transfer) permet à une application de communiquer avec une autre via HTTP.

☞ Elle utilise généralement :

- JSON comme format de données.
- Les méthodes HTTP (GET, POST, PUT, DELETE).

JSON:

Le JSON (JavaScript Object Notation) est un format de données très utilisé pour échanger des informations entre applications (ex : entre un serveur Laravel et un frontend).

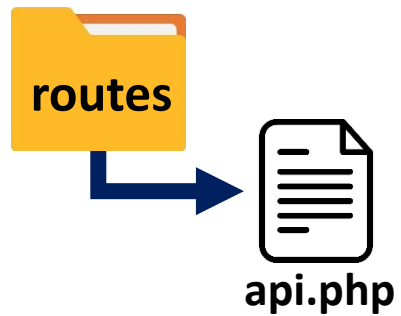
```
{  
  "nom": "Mansour",  
  "age": 30,  
  "ville": "Oujda"  
}
```

Structure REST standard:

Route	Action
GET /api/users	liste
GET /api/users/{id}	détail
POST /api/users	créer
PUT /api/users/{id}	update
DELETE /api/users/{id}	delete

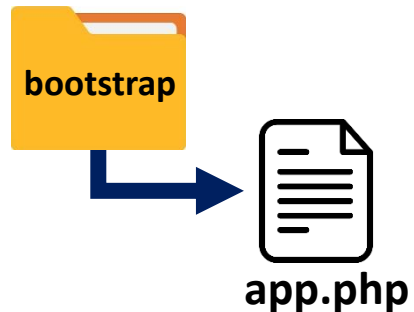
Routes API dans Laravel

- Les routes API sont définies dans :



```
Route::get('/users', [UserController::class, 'index']);  
Route::post('/users', [UserController::class, 'store']);  
Route::get('/users/{id}', [UserController::class, 'show']);  
Route::put('/users/{id}', [UserController::class, 'update']);  
Route::delete('/users/{id}', [UserController::class, 'destroy']);  
  
Route::apiResource('users', UserController::class);
```

- La modification du fichier **app.php** :



```
->withRouting(  
    web: __DIR__.'/../routes/web.php',  
    api: __DIR__.'/../routes/api.php',  
    commands: __DIR__.'/../routes/console.php',  
    health: '/up',  
)
```

Controllers API

```
php artisan make:controller Api/UserController --api
```

Codes HTTP importants

Code	Signification
200	OK
201	Créé
404	Non trouvé
401	Non autorisé
500	Erreur serveur

Réponses JSON

```
return response()->json([  
    'status' => true,  
    'data' => $users  
]);
```

```
return response()->json([  
    "message" => "Succès"  
], 200);
```

Principaux types d'authentification

1. Session (Web)

- Utilise des **cookies**
 - L'utilisateur se connecte → session stockée côté serveur
- ☞ Utilisé pour les applications web classiques

2. Token (API)

- Chaque utilisateur a un token unique
 - Envoyé dans chaque requête (Bearer Token)
- ☞ Simple et adapté aux API

3. JWT (JSON Web Token)

- Token contenant des informations (payload)
 - Stateless (pas de session côté serveur)
- ☞ Utilisé pour API modernes

4. OAuth (Login externe)

- Authentification via un service externe
- ☞ Exemple : connexion avec Google

Définition

Laravel Sanctum est un package officiel de Laravel qui permet de gérer l'authentification des utilisateurs, principalement pour les API, en utilisant des tokens simples et sécurisés.

Installation

```
composer require laravel/sanctum
php artisan migrate
```

Génération de token

```
$user = User::find(1);
$token = $user->createToken('api-token')->plainTextToken;
```

Configuration

Dans ***app\models\user.php***

```
use Laravel\Sanctum\HasApiTokens;

class User extends Authenticatable
{
    use HasApiTokens;
}
```

```
php artisan vendor:publish --
provider="Laravel\Sanctum\SanctumServiceProvider"
```

Route protégée

```
middleware('auth:sanctum')
```

Envoi du token (Postman)

```
Authorization: Bearer YOUR_TOKEN
```

8. API REST

Mini TP :

Objectif :

- Mettre en place une API REST et Sanctum avec Laravel
- Créer des routes API
- Utiliser un contrôleur API
- Retourner des réponses JSON
- Tester les requêtes API avec Postman

Etapas :

- Créer modèle + migration ***Product***
- Créer contrôleur API ***ProductController***
- Définir routes dans ***api.php***
- Ajoûlémenter CRUD (index, store, show, update, delete)
- Retourner des réponses JSON avec status codes
- Tester avec Postman

Partie 9 : Sécurité dans Laravel

9. Sécurité dans Laravel

Définition:

Le CSRF est une attaque où un attaquant exploite la session active d'un utilisateur pour exécuter des actions à sa place, sans son consentement.

👉 Exemple:

Un utilisateur déjà connecté visite un site malveillant, qui déclenche une requête vers une autre application où il est authentifié.

Vérification automatique:

Middleware utilisé :

```
App\Http\Middleware\VerifyCsrfToken
```

CSRF (Cross-Site Request Forgery)

Protection dans Laravel:

Laravel protège automatiquement contre CSRF via un token.

```
<form method="POST" action="/profile">
    @csrf
    <input type="text" name="name">
    <button type="submit">Envoyer</button>
</form>
```

@csrf génère un token sécurisé.

Important:

- Obligatoire pour toutes les requêtes POST, PUT, DELETE
- Non requis pour les API (souvent remplacé par Token)

Pourquoi ?

Ne jamais stocker les mots de passe en clair.

☞ Utiliser un hash sécurisé

Bonnes pratiques

Ne jamais décrypter un mot de passe

Toujours utiliser ***Hash::make()***

Laravel utilise :

- Bcrypt (par défaut)
- Argon2

```
use Illuminate\Support\Facades\Hash;  
  
$password = Hash::make('123456');
```

```
Hash::check('123456', $user->password);
```

Définition :

Injection de code SQL malveillant via les entrées utilisateur.

☞ Exemple dangereux :

```
DB::select("SELECT * FROM users WHERE email = '$email'");
```

Protection Laravel :

```
DB::table('users')  
    ->where('email', $email)  
    ->get();
```

```
User::where('email', $email)->first();
```

À éviter :

- concaténation SQL
- requêtes brutes non sécurisées

Définition :

Injection de scripts JavaScript dans une page web.

Protection Laravel :

```
{{ $userInput }}
```

Bonnes pratiques :

- Toujours utiliser `{{ }}`
- Valider les entrées utilisateur